

Pitch Aggregation — Differences & Agreements

Purpose. Three architecture-proposal pitches describe the same system from different angles. This memo surfaces every point of agreement, near-agreement, and divergence across them so each item can be closed with an explicit decision or scheduled for a discussion.

Source of truth. CRAFT-pitch.pdf (abbreviated **CRAFT**). The other two are read as deltas from it: - CRAFT-pitch-research-assistant.pdf → **RA** (CRAFT Research Assistant, Phase 1 prototype, Oct 2026) - CDD_architecture (1).pdf → **CDD** (CSA_DATA_DIGGER prototype + proposed Azure path)

Citations. CRAFT §4 = CRAFT pitch section 4. RA Fig 5 = Research Assistant pitch figure 5. CDD §3 = CSA_DATA_DIGGER section 3. — = silent (pitch does not address it).

Status tags. - **Aligned** — the three pitches agree in substance (phrasing may differ). - **Near-aligned** — same intent / goal, but mechanics, maturity (planned vs built), or vendor choice differ. Usually no exec call, just confirmation. - **Divergent** — clear disagreement; executive must choose. - **Discussion** — not a simple yes/no; needs a working session (scope trade-off, philosophy, missing information).

How to read. Skim the matrix top-to-bottom for the Status column. For every row that is not **Aligned**, a four-to-five-line narrative sits in §3 under the same ID. The matrix row ID (e.g. H3) is the anchor.

1. Where we stand

- **Aligned (settled):** 7 rows — citation verification, approved-sources-only enforcement, internal corpus access, separation of duties, branch/resume from log, dependency-footprint discipline, bilingual intent.
- **Near-aligned (same goal, confirm mechanics):** 16 rows — mostly identity/RBAC/HITL/audit mechanics and Azure-region posture.
- **Divergent (executive call required):** 14 rows — language stack, agent framework, agent autonomy, retrieval engine, isolation model, HITL reviewer UI location, cost-calculation philosophy, scope rows (risk classifier, vendor roll-up, external connectors).
- **Discussion (scope / philosophy):** 10 rows — shell-access change request, offline-vs-cloud trade-off, requirements-analysis / metadata-browser / manual-search feature pull-ins, external callers surface, CI-invariants adoption, backup-approver timeout.

The three pitches already agree on **what** CRAFT exists to do (cited retrieval over approved CSA documents with HITL on risky actions, bilingual, deployed in a Canadian PBMM region). They disagree principally on **what to build it in** (Python vs TypeScript vs scripted RAG), **how autonomous the agent should be**, **where cost calculations live** (LLM tool vs deterministic math), and **what scope Phase 1 should cover**.

2. Decision matrix

Legend: cells show the position + inline doc reference. Status in column 5. Action in column 6. Row IDs anchor the narrative in §3.

A. Platform & language

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
A1	Backend language / web framework	Flask + Dash, Python (§4)	Node.js + Express, TypeScript (§6)	Local Python web app, unspecified fw (§1)	Divergent	Decide: Python vs TypeScript stack
A2	Frontend / reviewer UI	Dash reviewer UI (§2b, §4)	React + assistant-ui (§6, Fig 2)	Web app, framework unspecified (§2)	Divergent	Decide: Dash vs React
A3	Single language end-to-end	Python throughout (§8 deps)	TypeScript throughout (§6)	—	Divergent	Follows from A1

B. Agent & control model

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
B1	Agent framework	Microsoft Agent Framework, ChatAgent (§4)	pi TypeScript (§6)	None — scripted RAG (§2)	Divergent	Decide: Agent Framework vs pi vs no-agent
B2	Agent autonomy	Bounded iteration + orchestrator (§2c PanelA, §5)	One autonomous agent, free to plan (§3, Fig 3)	Fixed pipeline, no agent (§2)	Divergent	Discussion: how autonomous
B3	Multi-agent vs single	Orchestrator + specialist agents (§5)	One agent per conversation (§2)	n/a	Divergent	Decide: multi-agent now or later
B4	Tool-gate / checkpoint	@tool (approval_mode) → ApprovalQueue (§2c PanelA, §4)	Checkpoint before every tool call (§3, Fig 3)	n/a	Near-aligned	Confirm: same mechanism, different names
B5	Bounded iteration / loop breaker	Bounded iteration cap (§4)	“Next” phase — budgets, step limits, loop breaker (§8, §9)	n/a	Near-aligned	Confirm: RA to match CRAFT’s maturity

C. LLM & retrieval

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
C1	LLM provider / region	Azure AI Foundry, Canadian PBMM (§4)	OpenAI GPT-5 on Azure AI Foundry, Canadian (§6, §5)	Local LLM today, planned Azure (§1, §3)	Near-aligned	Confirm: all three land on Foundry; pick model
C2	Retrieval engine	Azure AI Search hybrid (§4)	Orama hybrid today, managed later (§6, §9)	FAISS cosine + reranker local (§2)	Divergent	Decide: managed AI Search vs self-hosted vs FAISS
C3	Embedding model	Via Azure AI Search (§4)	OpenAI text-embedding-3 on Foundry (§6)	Local embedder (§2)	Near-aligned	Confirm: pick embedding model
C4	Document-preparation stack	pdfplumber, python-docx (§8)	mammoth / pdf.js / ExcelJS + content fingerprints (§6, Fig 2)	Chunker + FAISS at ingest (§2)	Divergent	Decide: Python vs TS libs; adopt fingerprinting?
C5	Citation verification	DocumentSearchTool returns doc/page/section (§5)	Runtime verifies doc/version/location exists (§3, Fig 4 steps 7-8)	Source attribution per answer (§2)	Aligned	None (settled)
C6	Confidence scoring	RAGAS-style, 0.6 threshold escalates (§5)	Planned — evidence + citation + agreement (§8, §9)	Confidence displayed per answer (§2)	Near-aligned	Confirm: adopt CRAFT's 0.6-escalate rule
C7	Grounding node	Explicit between retrieve and respond (§5)	Implicit via citation checker (§3)	—	Near-aligned	Confirm: make grounding explicit

D. Isolation & sandbox

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
D1	Code-exec sandbox	Azure Container Apps (§4)	Docker per conversation (§6, §7)	—	Divergent	Decide: ACA vs per-conv Docker
D2	Session isolation model	Managed DB session + WorkspaceStore (§4)	Docker container per conversation, removed on close (§7)	None discussed (§2)	Divergent	Decide: session-in-DB vs session-in-container
D3	Network posture inside sandbox	Isolated container (§4)	No network, read-only files, fixed CPU/mem (§5, §7)	n/a	Near-aligned	Confirm: adopt RA's explicit posture
D4	Shell access policy	No (only code_exec tool) (§2c PanelB)	Formal change request to permit shell inside sealed container (§10)	—	Discussion	Schedule discussion: §10 CR (see §4)

E. Identity, RBAC, HITL

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
E1	Identity provider	Entra ID + MSAL (§4)	Planned CSA sign-in via app backend (§7, Fig 1 dashed)	Missing today — “highest priority” (§3)	Near-aligned	Confirm: Entra ID is the target
E2	Role model	permissions.json, base/power/admin (§2b, §5)	Planned CSA roles via backend (§7)	Credentials + permissions, undefined (§3)	Near-aligned	Confirm: CRAFT's three-tier model
E3	Separation of duties	RBAC + classification tags (§5)	Requester never approves own request (§7)	—	Aligned	None (settled)
E4	HITL approval queue backing store	Postgres via psycpg[binary] (§4)	Persistent store, survives restart; vendor unnamed (§4, Fig 5)	—	Near-aligned	Confirm: Postgres as backing store
E5	HITL reviewer UI location	Dash reviewer UI (§4)	Approve/deny cards inside assistant-ui (§4, Fig 5)	—	Divergent	Decide: separate reviewer app vs cards in chat UI
E6	Backup / timeout path on HITL	—	No reply at 24h → backup approver (§4, Fig 5)	—	Discussion	Discuss: adopt 24h fallback into CRAFT

F. Audit & observability

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
F1	Audit sink	HashChainedJsonLog SHA-256 → Azure Log Analytics (§4)	pi append-only session log → Azure immutable storage (§2 Fig 2, §5)	—	Near-aligned	Confirm: hash-chain + Log Analytics
F2	Branch / resume from log	Resume route after HITL (§2c PanelA)	Supports branch & resume (§2 Fig 2)	—	Aligned	None (settled)

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
F3	Trace envelope	AgentAction envelope (§6)	Per-step record (message / tool call / result) (§2)	—	Near-aligned	Confirm: adopt CRAFT's envelope schema

G. Data sources & external connectors

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
G1	Internal corpus access	LocalFileSource (§2b, §2c PanelB)	Document library, read-only (§2)	All docs under configured root (§2)	Aligned	None (settled)
G2	SharePoint / SAP / STK / MATLAB / AI Search	Explicit {I} DataSource classes (§2b, §3, §7)	Phase 1 scope: approved docs only (§1)	—	Divergent	Decide: Phase 1 scope includes external connectors?
G3	Approved-sources-only enforcement	Classification tags at data store (§5)	"No hidden outside services" (§7)	Offline / local only (§1)	Aligned	None (settled)
G4	Simulated / classification labels	Classification tags enforced at store (§5)	"Simulated" badge on source cards (§8)	—	Near-aligned	Confirm: adopt visible badge on top of tags

H. Capability scope

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
H1	Bilingual EN/FR	Azure Translator + Foundry NLG (§3, §5)	Implicit via model (not called out)	Explicit / chat EN+FR (§2)	Near-aligned	Confirm: translator in loop vs model-only
H2	Risk classifier (UC-E2)	ClassifierTool, HITL gated (§3)	—	—	Divergent	Decide: Phase 1 includes UC-E2?
H3	Parametric cost estimation (UC-F1)	HistoricalMissionTool + CostAggregatorTool via LLM tool call (§3)	—	Deterministic math over dedicated DB; "LLM must not perform cost calculations" (§3)	Divergent	Major decision: LLM-driven cost vs deterministic math
H4	Vendor cost roll-up (UC-F4)	SAP DataSource + VendorAggregatorTool (§3)	—	—	Divergent	Decide: Phase 1 includes UC-F4?
H5	Report generation	Bilingual, HITL-gated, cited (§5)	—	/ report_generation with templates, uses LLM (§3)	Near-aligned	Confirm: adopt CRAFT's HITL+citation gate
H6	Requirements analysis	—	—	/ search / chat_bot finds missing/inconsistent/ duplicate reqs (§3)	Discussion	Discuss: pull into CRAFT?
H7	Document metadata browser	—	—	/ document_metadata filterable by mission/subject (§2)	Discussion	Discuss: pull into CRAFT?
H8	Manual search with page previews	—	—	/ manual returns page images; no LLM (§2)	Discussion	Discuss: pull into CRAFT?

I. Deployment & SLA

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
I1	Prototype target	Goes straight to prod shape	Single Azure VM (§5)	Local offline Linux + Windows (§1)	Divergent	Decide: unified prototype target
I2	Production target	App Service PBMM + SSC LaunchPad HA (§4)	App Service + Container Apps + Protected B landing zone (§5, Fig 6)	"Move to Azure", unspecified services (§3)	Near-aligned	Confirm: App Service PBMM + ACA
I3	Edge security	APIM + App Gateway (WAF, rate limit) (§4)	—	—	Discussion	Discuss: adopt APIM+WAF Phase 1
I4	HA / scale	Autoscale + Azure Front Door (§5)	Managed Azure services (§5)	—	Near-aligned	Confirm: AFD + autoscale
I5	Offline-capable operation	No (cloud-only)	No (cloud-only)	Yes — airgapped Linux+Windows today (§1)	Discussion	Discuss: do we lose offline on Azure migration?

J. Extensibility & QA

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
J1	Protocol / plug-in model	7 named protocols (Tool, DataSource, ...) (§4, §6)	Implicit building blocks (§2)	—	Divergent	Decide: formalise protocols

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
J2	“Add tool/connector = one class + one JSON row”	Yes (§2c PanelB)	Approved tools list, extensible (§3)	—	Near-aligned	Confirm: adopt one-class-one-row pattern
J3	CI invariants	pytest + pyright strict + ruff + bandit + pip-audit + detect-secrets + architecture tests (§4, §8)	—	—	Discussion	Discuss: adopt CRAFT’s CI suite
J4	Experimental-feature flags + contributor on-ramp	Flags gate experimental features; non-software teams land Claude skills on feature branches (§2c PanelC)	—	—	Discussion	Discuss: adopt flag + contributor model

K. Interoperability

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
K1	External token-bearing callers	Any token-bearing caller (user/cron/agent) hits CRAFT; webhook callbacks (§2b, §5)	—	—	Discussion	Discuss: external caller surface in Phase 1
K2	REST surface as integration contract	Explicit (§4, §5)	—	—	Discussion	Discuss: commit REST as contract
K3	Optional GraphQL layer	Noted as optional on top of REST (§5)	—	—	Discussion	Discuss: GraphQL need

L. Dependencies & external blockers

ID	Topic	CRAFT (SoT)	RA	CDD	Status	Action
L1	Dependency footprint discipline	Named list, “one targeted dep per row” (§8)	Named per layer (§6)	Local models only (§1)	Aligned	None (settled)
L2	External teams required	IT, Finance, Engineering, Domain (§7)	CSA identity provider team (§5)	TBD (§3)	Near-aligned	Confirm: CRAFT dependency table is canonical

3. Per-topic narrative

Rows marked **Aligned** are omitted — matrix line is sufficient.

A1 — Backend language / web framework

CRAFT. Flask + Dash, Python, routes are the integration contract (§4). **RA:** Node.js with Express, TypeScript end-to-end (§6). **CDD:** local Python web app, framework unspecified (§1). **Status:** Divergent — the single highest-leverage decision; cascades into A2, A3, B1, C4, J1. **Action:** Decide the primary stack (Python/Flask or TypeScript/Node); every other tooling choice flows from this.

A2 — Frontend / reviewer UI

CRAFT. Dash reviewer UI, shares auth and audit perimeter with Flask (§4). **RA:** React + `assistant-ui`, approve/deny cards built into the chat UI (§6, Fig 2, Fig 5). **CDD:** web app, framework unspecified (§2). **Status:** Divergent — tied to A1 but separable (a Python backend can serve a React UI). **Action:** Decide whether the HITL reviewer is a separate Dash surface or lives as cards inside the chat UI.

A3 — Single language end-to-end

CRAFT. Python stack (§8 dependency list). **RA:** “One language from the browser to the agent” (§6 — explicit value statement). **CDD:** silent. **Status:** Divergent — follows from A1. **Action:** None once A1 is decided.

B1 — Agent framework

CRAFT. Microsoft Agent Framework (`agent-framework`, Microsoft’s enterprise successor to Semantic Kernel), `ChatAgent` + `AzureOpenAIChatClient` (§4). **RA:** `pi` TypeScript framework; built for autonomous agents, writes an append-only log of every message/tool-call/result (§6). **CDD:** no agent framework — a scripted RAG pipeline (§2). **Status:** Divergent — the second-highest-leverage decision after language. **Action:** Decide between Agent Framework, `pi`, or no-framework/scripted. The three options give very different control surfaces.

B2 — Agent autonomy

CRAFT. Bounded iteration cap on an agent that may orchestrate specialist agents; approvals via `@tool(approval_mode)` (§2c PanelA, §5). **RA:** One autonomous agent per conversation, “free to plan its own research, but every action passes checkpoints” (§3, Fig 3). Explicitly cites UMR-094 as supporting agent autonomy. **CDD:** no agent; fixed pipeline (§2). **Status:** Divergent — this is

a philosophical as much as technical choice. **Action:** Hold a working session on how autonomous the agent is allowed to be in Phase 1 (fixed script, bounded loop, or free planner).

B3 — Multi-agent vs single

CRAFT. Primary ChatAgent + orchestrator + specialist agents via framework multi-agent primitives (§5). RA: Explicit “one agent per conversation” (§2). CDD: n/a. Status: Divergent. **Action:** Decide whether multi-agent is in Phase 1 scope or deferred.

B4 — Tool-gate / checkpoint

CRAFT. @tool(approval_mode="always_require") → ApprovalQueue → resume route (§2c PanelA, §4). RA: Explicit “Checkpoint” node — every tool request passes through it (tool on approved list? within budget? high-risk? record decision) (§3, Fig 3). CDD: n/a. Status: Near-aligned — same mechanism, different names. **Action:** Confirm the mechanism is identical; standardize naming in the implementation.

B5 — Bounded iteration / loop breaker

CRAFT. Bounded iteration cap (§4). RA: Listed under “Next” phase — approved tools, budgets, loop breaker (§8, §9). CDD: n/a. Status: Near-aligned — RA plans to catch up to CRAFT. **Action:** Confirm iteration cap is Phase-1 gated, not deferred.

C1 — LLM provider / region

CRAFT. Azure AI Foundry (model routing + rate limits + Content Safety) in Canadian PBMM region (§4). RA: OpenAI GPT-5 on Azure AI Foundry, Canadian region in production (§6, §5). CDD: Local LLM today; planned move to “more performant Azure models” (§1, §3). Status: Near-aligned — all three converge on Foundry in a Canadian region. **Action:** Pick the model (GPT-5 vs Foundry’s model-routing default) and confirm Content Safety is on.

C2 — Retrieval engine

CRAFT. Azure AI Search hybrid (vector + BM25) via azure-search-documents, exposed as {I} DataSource (§4). RA: Orama (hybrid, in-process TypeScript) today with 0.7/0.3 weighted ranking; moves to a managed service in production — “Azure AI Search is one option” (§6, §9). CDD: FAISS cosine similarity + reranker + top-5 to LLM, local (§2). Status: Divergent. **Action:** Decide between managed Azure AI Search (CRAFT), self-hosted in-process index (RA), or FAISS (CDD). Note RA already points to AI Search as its production fallback.

C3 — Embedding model

CRAFT. Covered implicitly by Azure AI Search (§4). RA: OpenAI text-embedding-3 on Azure AI Foundry (§6). CDD: Local embedder, unspecified (§2). Status: Near-aligned — all three use embeddings; model unnamed in CRAFT. **Action:** Confirm text-embedding-3 (or AI Search’s default) and name it in CRAFT.

C4 — Document-preparation stack

CRAFT. pdfplumber, python-docx listed in dependency surface (§8). RA: mammoth (Word), pdf.js (PDF), ExcelJS (Excel), with a content fingerprint per prepared document — the thing that makes citations verifiable (§6, Fig 2). CDD: Chunk + FAISS at ingest, SQLite stores chunks + metadata (§2). Status: Divergent — tied to A1 for library choice; fingerprinting is a separate design decision. **Action:** Decide whether to adopt RA’s fingerprinting discipline regardless of library choice.

C6 — Confidence scoring

CRAFT. Grounding + RAGAS-style scoring; threshold 0.6 escalates to reviewer (§5). RA: Planned (Phase 1 roadmap) — from evidence coverage + citation checks + agreement between sources (§8, §9). CDD: Confidence level displayed per answer (§2) — not threshold-gated. Status: Near-aligned — all three show confidence; only CRAFT routes low scores. **Action:** Adopt CRAFT’s 0.6-escalate rule as the standard.

C7 — Grounding node

CRAFT. Explicit grounding node between retrieve and respond (§5). RA: Implicit — the citation checker enforces grounding at the output boundary (§3). CDD: Silent. Status: Near-aligned. **Action:** Confirm an explicit grounding step is in the agent loop, even when the citation checker is at the output.

D1 — Code-exec sandbox

CRAFT. Azure Container Apps sandbox via CodeExecTool (§4). RA: Docker container per conversation — the conversation’s tools AND session run inside it; no network, read-only files (§6, §7). CDD: No sandbox discussed. Status: Divergent. **Action:** Decide: Azure Container Apps (one shared sandbox, per-call container) or per-conversation Docker (one container for the whole session).

D2 — Session isolation model

CRAFT. Session + per-user data in Managed DB behind `WorkspaceStore`, RBAC-scoped (§4). RA: Each conversation lives inside its own Docker container; container removed on close (§7). CDD: No session isolation. Status: Divergent — different models of what “a session” is. **Action:** Decide session model: DB-backed workspace vs ephemeral container.

D3 — Network posture inside sandbox

CRAFT. Isolated container (§4). RA: Explicit — no network access, read-only system files, no admin, fixed CPU/mem/process limits (§5, §7). CDD: n/a. Status: Near-aligned. **Action:** Adopt RA’s explicit posture statement as the acceptance criteria for CRAFT’s sandbox.

D4 — Shell access policy

CRAFT. No shell; only the sandboxed `code_exec` tool (§2c PanelB). RA: Formal change request (§10) to amend §1.3, UMR-094 and ASG-012 so the agent may use a shell inside its sealed container, with named search tools kept alongside. Industry evidence: Claude Code (Anthropic) and Cursor both report better results with shell than with vector search alone. CDD: Silent. Status: Discussion — this is a change request against the requirements doc, not a design choice. See §4. **Action:** Schedule the §10 CR review (see §4 for conditions).

E1 — Identity provider

CRAFT. Entra ID + MSAL behind `{I} AuthProvider` (§4). RA: Planned CSA sign-in via the app backend; dashed in Fig 1 (not yet built) (§7). CDD: No auth today — flagged as the “highest priority missing capability” (§3). Status: Near-aligned. **Action:** Confirm Entra ID + MSAL as the target; RA and CDD align to it.

E2 — Role model

CRAFT. `permissions.json` with three role levels (base < power < admin) + route rules (§2b, §5). RA: Planned CSA roles managed by backend (§7). CDD: “User access restricted based on credentials and permissions” — unspecified (§3). Status: Near-aligned. **Action:** Confirm CRAFT’s three-tier model as canonical.

E4 — HITL approval queue backing store

CRAFT. Postgres via `psycopg[binary]` behind `{I} ApprovalQueue`; durable across restarts, multi-reader (§4). RA: Persistent store that survives restarts — vendor unnamed (§4, Fig 5). CDD: None. Status: Near-aligned. **Action:** Confirm Postgres as the backing store across all pitches.

E5 — HITL reviewer UI location

CRAFT. Dedicated Dash reviewer UI consuming the queue (§4). RA: Approve/deny cards built into the `assistant-ui` library — reviewer and user share the same UI (§4, Fig 5). CDD: None. Status: Divergent. **Action:** Decide: separate reviewer app (CRAFT) or in-chat cards (RA). Impacts role separation UX.

E6 — Backup / timeout path on HITL

CRAFT. Silent. RA: After 24h with no reply, a backup approver is notified (§4, Fig 5). CDD: Silent. Status: Discussion. **Action:** Discuss adopting RA’s 24h fallback and defining the backup approver per role.

F1 — Audit sink

CRAFT. `HashChainedJsonLog` writing append-only JSONL with SHA-256 hash chain, daily rotation, forwarded to Azure Log Analytics for KQL + immutable retention (§4). RA: `pi`’s append-only session log, forwarded to Azure immutable storage in production (§2 Fig 2, §5). CDD: Silent. Status: Near-aligned — intent matches, mechanics differ (hash-chain is CRAFT-specific). **Action:** Confirm hash-chain + Log Analytics as the standard; RA to adopt hash-chain.

F3 — Trace envelope

CRAFT. `AgentAction` envelope: a single schema wraps every tool call and LLM turn (§6). RA: Per-step record of every message, tool call, and result (§2). CDD: Silent. Status: Near-aligned. **Action:** Confirm CRAFT’s envelope schema is canonical.

G2 — SharePoint / SAP / STK / MATLAB / Azure AI Search as DataSources

CRAFT. Each is a `{I} DataSource` class with MSAL / OAuth at the perimeter; one JSON row per source in `data_sources.json` (§2b, §3, §7). RA: Phase 1 scope is “approved document collection” only (§1); external connectors not in Phase 1. CDD: Silent. Status: Divergent — this is a scope decision. **Action:** Decide whether SAP / STK / MATLAB / SharePoint connectors land in Phase 1 or defer.

G4 — Simulated / classification labels

CRAFT. Classification tags enforced at the data store (§5). RA: Visible “simulated” badge on every source card that comes from simulated data (§8). CDD: Silent. Status: Near-aligned — different layer, complementary. **Action:** Adopt RA’s visible badge on top of CRAFT’s store-level tags.

H1 — Bilingual EN/FR

CRAFT. Azure Translator + Foundry NLG generate narrative sections; report generator attaches citations and runs HITL (§3, §5). RA: Not explicitly called out — assumed from model capability. CDD: Explicit — /chat supports both English and French (§2). Status: Near-aligned. **Action:** Confirm whether Azure Translator sits in the loop or model handles both languages natively.

H2 — Risk classifier (UC-E2)

CRAFT. ClassifierTool scores severity; HITL gates the result; writes to risk register (§3). Requirements: UMR-002/006, UMR-036-040, HITL-001. RA: Silent. CDD: Silent. Status: Divergent — scope decision; the other two pitches did not plan for this. **Action:** Decide whether UC-E2 is in Phase 1 or deferred.

H3 — Parametric cost estimation (UC-F1) — major decision

CRAFT. HistoricalMissionTool returns similar missions; CostAggregatorTool computes weighted distance; HITL approves; result written. All inside the agent’s tool-call loop (§3). RA: Silent. CDD: Explicitly rejects this model: “I do not believe an LLM should be responsible for performing cost calculations or generating authoritative cost analyses, as numerical accuracy is critical.” Proposes a /cost_estimation page using deterministic mathematical algorithms over a separate validated cost database; LLM used only to explain or summarize (§3). Status: Divergent — this is a philosophical split. **Action:** Hold a working session. The question is not implementation detail but principle: does the LLM compute cost (CRAFT) or only narrate deterministic math (CDD)?

H4 — Vendor cost roll-up (UC-F4)

CRAFT. SAP DataSource returns line items; VendorAggregatorTool sums by vendor; report returned. Requirements: UMR-053-055 (§3). RA: Silent. CDD: Silent. Status: Divergent — scope decision. **Action:** Decide whether UC-F4 is in Phase 1 or deferred.

H5 — Report generation

CRAFT. Bilingual (Translator + Foundry NLG), citations attached, runs through HITL (§5). RA: Silent. CDD: Dedicated /report_generation page: select type, apply template, LLM generates (§3). No HITL mentioned. Status: Near-aligned. **Action:** Adopt CRAFT’s HITL + citation gate even for templated reports.

H6 — Requirements analysis

CRAFT. Not a feature. RA: Not a feature. CDD: /search/chat_bot identifies missing requirements, inconsistencies, duplicates across selected documents (§3). Status: Discussion — only CDD proposes it. **Action:** Discuss whether this capability pulls into CRAFT (it would land as another {I} Tool).

H7 — Document metadata browser

CRAFT. Not a feature. RA: Not a feature. CDD: /document_metadata page filters by Mission, Subject area, Document type, extracted metadata (§2). Status: Discussion. **Action:** Discuss whether this UX lands in CRAFT.

H8 — Manual search with page previews

CRAFT. Not a feature. RA: Not a feature. CDD: /manual returns images of the pages the chunks came from; Next/Previous navigation; no LLM (§2). Status: Discussion. **Action:** Discuss whether manual (non-AI) search is in Phase 1.

I1 — Prototype target

CRAFT. Goes straight to the production shape (no separate prototype target). RA: Single Azure VM for the prototype; production moves each block to a managed Azure service (§5). CDD: Runs on offline Linux and Windows machines, deployed as a web app (§1). Status: Divergent. **Action:** Decide the unified prototype target before Phase 1 kick-off.

I2 — Production target

CRAFT. App Service PBMM + SSC LaunchPad HA (§4). RA: App Service + Container Apps + Protected B landing zone, Canadian region (§5, Fig 6). CDD: “Move to AZURE” — services unspecified (§3). Status: Near-aligned. **Action:** Confirm App Service PBMM + Container Apps + LaunchPad as the canonical landing.

I3 — Edge security

CRAFT. APIM + App Gateway with WAF and rate limit (§4). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss adopting APIM + WAF in Phase 1, or deferring to production.

I4 — HA / scale

CRAFT. Autoscale + Azure Front Door; SSC LaunchPad for landing zone; health checks + load-test harness (§5). RA: “Each block has a managed Azure home” (§5). CDD: Silent. Status: Near-aligned. **Action:** Confirm AFD + autoscale.

I5 — Offline-capable operation

CRAFT. Cloud-only. RA: Cloud-only. CDD: **Explicitly offline today** — RAG built entirely on local resources (LLM, embedder, reranker), works without internet connectivity, “maintaining full control over sensitive data” (§1). Status: Discussion — moving CDD to Azure sacrifices this capability. **Action:** Discuss whether CSA requires an offline fallback (airgapped lab, classified environments). If yes, Azure-first means losing a capability CDD has today.

J1 — Protocol / plug-in model

CRAFT. Seven named Python Protocols — `{I} Tool`, `{I} DataSource`, `{I} ChatProvider`, `{I} WorkspaceStore`, `{I} AgentActionLog`, `{I} ApprovalQueue`, `{I} AuthProvider`. Verified Pyright-strict (§4, §6). RA: Building blocks shown in Fig 2 but not formalized as protocols (§2). CDD: Silent. Status: Divergent — not conflicting but materially different maturity. **Action:** Decide whether formal protocols are a Phase 1 commitment.

J2 — “Add tool/connector = one class + one JSON row”

CRAFT. Yes, via kind registries (§2c PanelB). RA: Approved tools list; new tools can be added (§3). CDD: Silent. Status: Near-aligned. **Action:** Confirm the one-class-one-row pattern is adopted whatever the stack choice.

J3 — CI invariants

CRAFT. pytest + pyright strict + ruff + bandit + pip-audit + detect-secrets + doctest + architecture tests (route × role matrix, protocol conformance, audit-envelope invariants) (§4, §8). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss adopting CRAFT’s CI suite as the project standard; TypeScript equivalents would need to be chosen if A1 picks TS.

J4 — Experimental-feature flags + contributor on-ramp

CRAFT. Role flags gate experimental features for an opt-in cohort; non-software contributors land Claude skills, workflows, and rules on feature branches only, exercised behind a flag until they graduate (§2c PanelC). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss the governance implications before adopting (who approves graduation, how long a flag can live).

K1 — External token-bearing callers

CRAFT. Any token-bearing caller — user, cron, pipeline, agent — can hit any route; tools can carry webhook callbacks so long-running simulations post back (§2b, §5). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss whether external callers (automation, pipelines) are in Phase 1 or deferred.

K2 — REST surface as integration contract

CRAFT. Routes are the integration contract (§4, §5). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss committing REST as the public contract.

K3 — Optional GraphQL layer

CRAFT. Noted as optional on top of REST (§5). RA: Silent. CDD: Silent. Status: Discussion. **Action:** Discuss whether GraphQL is required or stays optional.

L2 — External teams required

CRAFT. Named table: IT, Finance, Engineering, Domain, each tied to a specific capability (§7). RA: CSA identity-provider team (§5). CDD: TBD (§3). Status: Near-aligned. **Action:** Confirm CRAFT’s dependency table is the canonical blocker list.

4. Open change-request against the requirements document

RA §10 — Permit shell access inside the sealed per-conversation container.

The requirements document today rules out free-form shell access for agents (§1.3, UMR-094, ASG-012). RA proposes amending those requirements, with the following argument and conditions:

- **Why.** Modern models are strong with `rg`, `find`, `sed`, `awk`; a fixed set of named tools can only offer combinations someone thought to build in advance. Industry evidence: Anthropic’s Claude Code (shell-first after a vector-DB attempt) and Cursor (grep-heavy, +12.5% accuracy with semantic search added on top).
- **What is already addressed.** The container has no network, read-only system files, no admin, fixed CPU/memory/process limits, read-only document mounts, and is removed when the conversation ends. Every command, its output and its result are written to the activity record. Hidden instructions in a document can only act inside the sealed container with nowhere to send data.
- **Proposed conditions.** (i) shell only inside the conversation’s own container; (ii) every command recorded and visible to the user; (iii) checkpoint inspects commands before they run and can deny them; (iv) hybrid keyword + semantic search remains the default search path per

UMR-004; (v) after Phase 1, compare answer quality with and without the shell on the acceptance data set (UMR-064) and decide whether to keep it.

- **Requested decision.** Submit a change through CSA’s document control process (§1.4) to amend §1.3, UMR-094 and ASG-012.

Action. Schedule the §10 CR review. The decision is binary (accept the amendment and run the Phase 1 comparison, or reject and keep the named-tool list). The decision interacts with H3 (if cost calculations stay LLM-driven, shell is more useful for cross-table checks).

5. One-pitch-only capabilities

Items raised by exactly one pitch; executives may want to pull them into the CRAFT baseline (or explicitly drop them).

- **CDD §3 — “LLMs must not perform cost calculations”** (deterministic math + validated cost models + dedicated DB). Mirrors H3. **Decision needed.**
- **CDD §1 — Fully offline operation** on Linux and Windows (local LLM/embedder/reranker, no internet). Mirrors I5. **Decision needed if any CSA environment requires airgapping.**
- **CDD §2 — /manual page-image previews** for direct human verification without AI. Mirrors H8.
- **CDD §2 — /document_metadata** filterable browser. Mirrors H7.
- **CDD §3 — Requirements-analysis flow** finding missing / inconsistent / duplicate requirements. Mirrors H6.
- **RA §10 — Shell inside sealed container** with industry evidence. Mirrors D4 and §4 above.
- **RA §4, Fig 5 — 24h backup approver** on no reply. Mirrors E6.
- **RA §2, Fig 2 — Content fingerprinting** of prepared documents to make citations verifiable. Mirrors C4; worth adopting regardless of library choice.
- **RA §8 — “Simulated” badge** on source cards. Mirrors G4.
- **CRAFT §4 — Experimental-feature flags + non-software-team contributor on-ramp** (Claude skills on feature branches, gated behind flags). Mirrors J4.
- **CRAFT §4, §8 — Full CI invariants suite** (pytest + pyright strict + bandit + pip-audit + detect-secrets + architecture tests). Mirrors J3.
- **CRAFT §2b, §5 — External token-bearing callers + webhook callbacks** for long-running simulations (STK/MATLAB posting back without blocking the agent). Mirrors K1.
- **CRAFT §5 — Optional GraphQL layer** on top of REST. Mirrors K3.

6. Terminology map

Same concept, different name — so the three pitches aren’t read as describing different systems.

Concept	CRAFT	RA	CDD
Agent runtime	Agent Framework ChatAgent (§4)	Agent runtime, p i (§6)	— (RAG pipeline, no agent) (§2)
Where tool approval happens	@tool (approval_mode) → ApprovalQueue (§4)	Checkpoint before every tool call (§3)	—
Where session state lives	{ I } WorkspaceStore on managed DB (§4)	Docker container per conversation (§7)	SQLite + FAISS at ingest (§2)
Append-only record of agent actions	HashChainedJsonLog → Log Analytics (§4)	p i append-only session log → immutable storage (§2 Fig 2)	—
Reviewer approve/deny surface	Dash reviewer UI (§4)	Cards inside assistant-ui (§4 Fig 5)	—
Retrieval contract	{ I } DataSource + AI Search (§2b, §4)	Document library (§2)	FAISS + reranker (§2)
Citation verifier	Grounding + RAGAS + 0.6 threshold (§5)	Citation checker before any source card is shown (§3, §4 Fig 4)	Source attribution + confidence on / chat (§2)
Approved-sources enforcement	Classification tags at store (§5)	“No hidden outside services” (§7)	Local-only operation (§1)
Report generator	Translator + Foundry NLG + HITL (§5)	—	/report_generation with templates (§3)
Identity	Entra ID + MSAL (§4)	Planned CSA sign-in via backend (§7)	Missing today (§3)